

Brain Blast Architecture

A client-only React 19 app. There is no server and no network. Each profile is one JSON document in `localStorage`, and every question is generated or looked up in the browser. It ships as a PWA and as a single self-contained `brainblast.html`.

React 19 + Vite 8

~70 JS modules

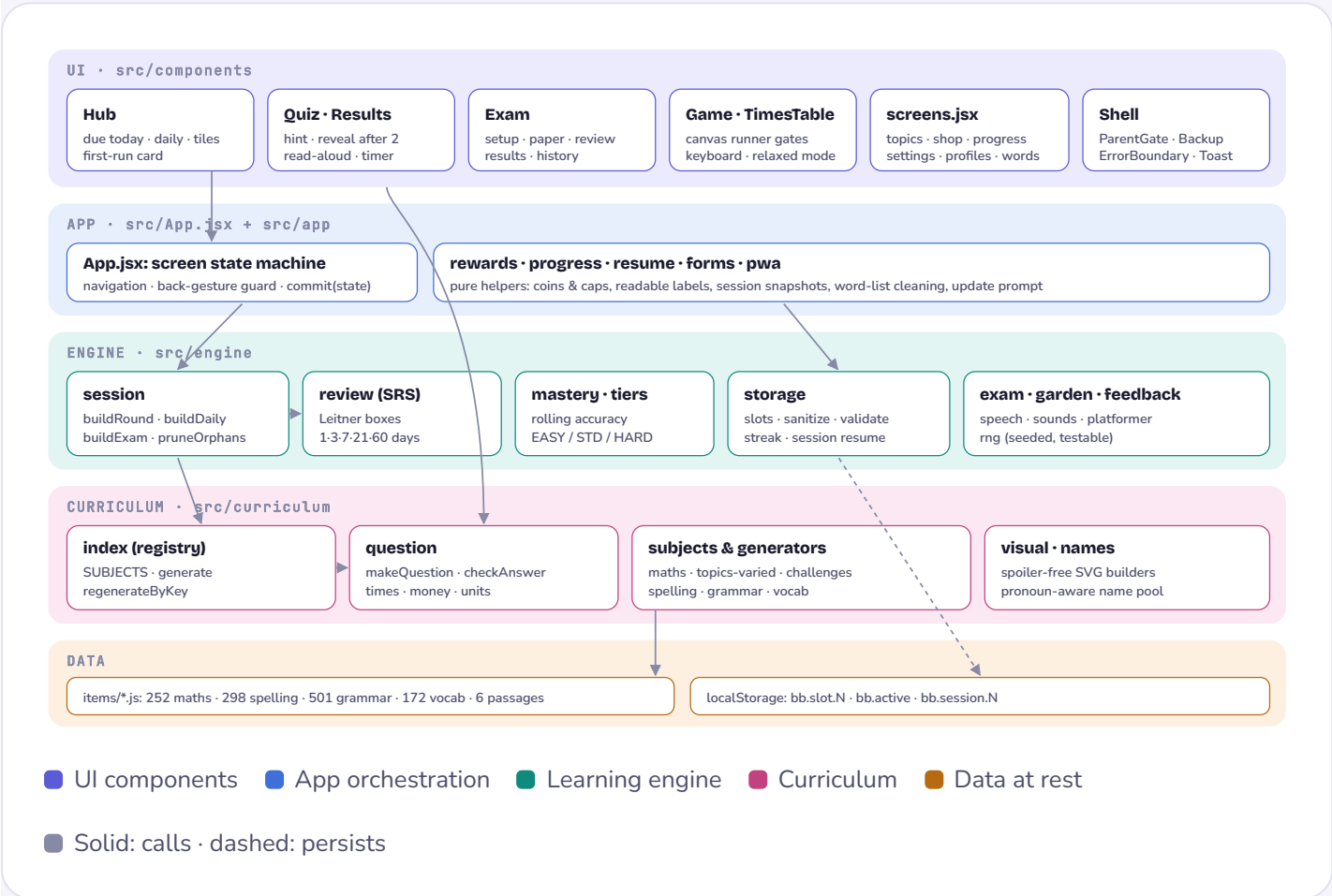
62 topics · 4 subjects

1,079 Vitest tests

0 runtime dependencies beyond React

System map

Four layers, and dependencies only point downward. The UI never touches `localStorage` or item data directly. It goes through the session builder, answer checking and storage.



One practice round, end to end

1

Hub → TopicPicker. The child picks Maths → Fractions. App calls `buildRound({subject, topic, reviewState, masteryState, size})`.

- 2 **Reviews first.** Due SRS keys are rebuilt with `regenerateByKey` . Keys that no longer exist, or that belong to another topic, are skipped before the cap of 4.
- 3 **Fresh questions.** Topics are weighted to the weakest, with a per-topic cap. Each topic's tier comes from mastery (Standard until 4 attempts) unless a parent has pinned one.
- 4 **Answering.** `checkAnswer(given, question)` canonicalises typing, then compares times, money, units and numbers structurally. Multiple choice is exact. After 2 misses the answer and "Why" are revealed.
- 5 **Grading.** Only the first try counts. `applyGrade` promotes an item only if it was due, and `recordAnswer` updates mastery. The session is snapshotted so a refresh can resume it.
- 6 **Finish.** `app/rewards` totals the coins, round bonus and streak reward, the garden is watered, the session is cleared, and Results shows the true total.

Module responsibilities

Question model

`curriculum/question.js` .

One shape for every subject: prompt, answer, options, hint, explain, visual, accept, speak, passage, tier, reviewKey. It also owns fair answer checking.

Registry (open/closed)

`curriculum/index.js` .

Subjects register their topics here. Adding a topic changes nothing else.

`regenerateByKey` means a review weeks later needs only its key.

Session policy

`engine/session.js` . The one place that decides what a round, daily challenge or exam contains: review share, passage clusters, topic spread, de-duplication.

Storage boundary

`engine/storage.js` . Every read passes through `sanitizeSave` , so a corrupt or hand-edited save can't crash the Hub. Restores use `validateSave` and a confirm that names the profile.

Pure App helpers

`src/app/*` . Rewards, progress labels, resume snapshots and form cleaning are pulled out of App.jsx, so they're unit-tested without a DOM.

Shared UI

`components/common.jsx` . TopBar, Tile, SpeakButton, ConfirmDialog (native `<dialog>`), PassagePanel and TierBadge. `engine/feedback.js` holds the hint and reveal rules.

Persistence schema

KEY	HOLDS	SAFETY
bb.slot.0-3	Profile: name, coins, review{key→box,due,...}, mastery{topic→attempts,correct}, streak, garden, inventory, customWords, settings, stats, examHistory (≤20), ttCoins	Type-guarded on every load; bad records dropped, counters repaired
bb.active	Active slot index	Defaults to 0
bb.session.N	Unfinished round or exam snapshot	Expires after 12 h; cleared on finish or delete
brainblast.save.v5	Legacy single-profile save	Migrated once into slot 0, then removed

box 0 · today

1 · +1d

2 · +3d

3 · +7d

4 · +21d

5 · +60d → mastered

Delivery & quality

Build targets

- `npm run build`: PWA with one manifest and a prompt-to-update toast
- `npm run bundle`: `brainblast.html`, fully inlined, opens with a double-click
- No web fonts or runtime fetches, so it's offline by construction

Tests (Vitest, node)

- Table-driven answer checking: 128 cases
- SRS, DST, storage validation, session invariants
- Sweeps of every topic × style × tier × many seeds: one correct option, no equivalent distractors, no float artefacts, pronoun agreement

Design rules

- Dependencies only point downward
- Pure functions in engine/curriculum; side effects at the edges
- Seeded RNG everywhere, so every question is reproducible